

Security Trust in Agent-Assisted Coding

Revised Research Plan and Progress Report

Working document for supervisor and proposal review

5 October 2026

Abstract

This proposed study examines how early-career developers judge and verify code produced with a coding agent. Each participant will complete one acceleration-encouraged and one exploration-encouraged JavaScript task. The study will compare security confidence with assessed code security, trace whether exposed SQL-injection and path-traversal weaknesses remain in final submissions, and examine security checks before and after a provisional keep decision. This report revises an earlier, broader trust-calibration proposal. It summarizes relevant evidence, states the present research questions, specifies a study and evaluation plan, and identifies questions the researcher must be able to defend. **No participant experiment or pilot has been run, and there are no preliminary results.** All task and measurement details below are proposed instruments awaiting a pilot and ethics approval.

1 Revised introduction, motivation, scope, aim, and objectives

1.1 Problem and motivation

Coding agents can implement a feature quickly, but a developer remains responsible for deciding whether a proposed change is safe to keep. A function may pass normal tests and still let untrusted input alter a database query or escape a permitted file directory. The issue is therefore not only whether an agent produces a weakness; it is whether a developer recognizes it, verifies the code, repairs it if needed, and submits a secure final feature.

Human-automation work describes appropriate trust as reliance that matches the system's actual capability [1, 2]. In coding, the relevant belief must be tied to a particular change. Perry et al. found that AI-assisted participants in their study produced less secure code overall and were more likely to believe their solutions were secure; in one JavaScript SQL task, SQL-injection-vulnerable solutions occurred in 36% of the assisted group and 7% of controls [3]. These figures motivate the confidence-security question but do not predict what will happen with a different agent, population, or task.

Security is a useful focus because a generic statement such as "I trust the agent" cannot show whether a developer recognized a concrete flaw. This study asks for confidence that an *exact proposed code snapshot* meets a stated security requirement, then checks that snapshot and the final repository independently. The aim is to understand the complete developer-agent decision path, including rejection and later repair.

1.2 Aim and objectives

Aim. Examine whether developers working with a coding agent in acceleration and exploration episodes differ in their security judgments, reliance on generated changes, verification, and trust calibration on security.

Operationally, "trust calibration" means the alignment of stated security confidence with assessed security of the *same* proposed change. The objectives are:

1. compare confidence–security alignment under two assigned task orientations;
2. trace exposed vulnerable proposals through keep/reject decisions, repairs, and final code;
3. describe security checks before and after provisional retention and relate them to final outcomes; and
4. examine whether these patterns differ for SQL injection and path traversal.

1.3 Scope and change from the original proposal

The February 2026 proposal asked broadly about acceptance speed, generic correctness, checking, automatic mode detection from IDE events, and different software-engineering task types. The present study narrows the outcome to *security confidence and named vulnerabilities*; moves from a quick acceptance proxy to final-code security; and treats observed mode as an interpretation supported by behavior and interviews, not as a clickstream diagnosis. It focuses on final-year computing students and junior engineers rather than an unrestricted experience range. The earlier idea of hidden insecure Markdown instructions is outside the current four research questions.

The study is limited to small JavaScript/Node tasks, two weakness classes, a controlled laboratory workflow, and an early-career cohort. It will not estimate how often current agents naturally generate these vulnerabilities, establish a general security rate for production code, or represent senior developers.

2 Literature survey and research gap

2.1 Trust, context, and interaction mode

Lee and See frame trust as a guide to reliance under uncertainty [1]; Okamura and Yamada discuss calibrating trust to changing automation capability [2]. These are theoretical foundations, not evidence that a specific coding mode causes insecure code. Wang et al. found that trust in AI code-generation tools depends on the developer, task, and interaction context [4]; Brown et al. identify properties of both suggestions and users that influence trust in code completion [5]. Acceptance alone is therefore an incomplete security measure.

Barke et al.'s grounded study of 20 programmers describes *acceleration*: the programmer knows the next step and uses the assistant to proceed faster; and *exploration*: the programmer is unsure how to proceed and uses it to find options [6]. These are accounts of developer intention, not a validated two-state classification algorithm. This study will randomly assign task *encouragement* and separately code observed interaction episodes. If the two do not align, the manipulation has not established a cognitive-mode effect.

2.2 Security outcomes with AI coding assistance

Prior results are mixed. Perry et al. reported less secure solutions and greater perceived security under AI assistance in their tasks [3]. Sandoval et al.'s randomized C-programming study with 58 students found only a small security difference in its setting [7]. In a 25-participant Copilot study, Asare et al. reported more secure solutions with access to Copilot on harder tasks and no effect on an easier task [8]. These studies justify testing context and task conditions rather than assuming that AI assistance always increases risk.

Controlled insecure assistance has also been studied. Oh et al. examined developers given insecure suggestions from poisoned AI models [9]. Serafini et al. tested warnings, security prompts, and guidance in a manipulated ChatGPT task [10]. Such work supports deliberately ensuring that participants

encounter a known flaw, provided the researcher reports how each stimulus was generated or edited. It does not establish that a hidden repository instruction is the right manipulation for the present RQs.

2.3 Review behavior and the closest recent study

Khalid et al.’s 2026 preprint is especially close: 100 developers evaluated security-varying AI suggestions for C linked-list tasks, selected and edited code, and submitted final solutions; 23 were interviewed [11]. Many checks were limited, but confidence in final-code security was associated with actual security. This is an important counterexample to the claim that developers are always overconfident. The present study should not claim to be the first security evaluation of AI-generated code. Its proposed contribution is the paired acceleration/exploration encouragement comparison, exact proposal-level probability judgments, time-ordered checks around provisional retention, and two JavaScript weakness classes in an agent workflow.

Tang et al. show that validation and repair can be studied with IDE actions, observation, and interviews [12]. Mozannar et al. used screen replay and retrospective labels to model activities in AI-assisted programming [13]; Wu et al. studied intention, action, tool use, and emotion with 76 developers using retrospective screen-based labels [14]. Their taxonomies differ from Barke’s modes. They support *triangulation*, not the assertion that an extension alone can read a developer’s mind.

2.4 Why SQL injection and path traversal?

MITRE’s 2024 CWE Top 25 ranked SQL injection (CWE-89) third and path traversal (CWE-22) fifth [15]. These are serious and technically distinct source-to-sink errors: request input can alter SQL syntax, or a path can escape an allowed directory. Both can be exercised with small, local JavaScript applications and checked with functional tests, attack tests, and code review. MITRE’s list is *not* a survey of LLM-generated vulnerabilities. The defense is importance, contrasting mechanisms, and testability—not a claim that these are the two most frequent AI failures.

2.5 Gap and proposed contribution

Existing studies establish that AI-assisted security outcomes can vary, insecure suggestions can be experimentally supplied, and evaluation behavior can be observed. The narrower gap is a linked account of *assigned task orientation*, *observed mode*, *confidence about an exact agent proposal*, *verification timing*, and *final vulnerability retention* in the same participant. If successfully piloted and completed, the study can contribute a reproducible task/oracle set and evidence on where security review succeeds or fails. Its novelty is this combination and comparison, not the individual concepts.

3 Revised research questions

- RQ1.** How does alignment between developers’ security confidence and assessed code security differ between acceleration-oriented and exploration-oriented conditions? How does this relate to observed interaction mode?
- RQ2.** How do the two conditions affect whether security vulnerabilities remain in the final submitted code?
- RQ3.** How do security verification behaviors before and after provisional retention of agent-generated code differ between acceleration-encouragement and exploration-encouragement conditions, and how are these behaviors associated with removal or retention of known exposed vulnerabilities in the final submission?
- RQ4.** Do these patterns differ between SQL injection and path traversal opportunities?

For RQ2, the evidence includes exposed vulnerabilities, participant decisions, repairs, and final code security. The RQs are non-directional. Exploration may encourage checks because the developer is uncertain; it may also encourage reliance on an unfamiliar API. Results in either direction, or a bounded null result, are informative. For causal wording, the primary comparison is between *randomly assigned encouragement packages*. Observed mode is coded after the fact and analyzed descriptively; it is not itself randomized.

4 Research approach, methodology, and design

4.1 Design and participants

The proposed study is a within-participant, counterbalanced experiment with qualitative follow-up. Each participant completes two agent-assisted tasks: one with acceleration encouragement and one with exploration encouragement. Project A and Project B are each used under both conditions across participants. The four project-condition-order combinations are assigned approximately evenly, so project identity and first-task learning do not define the condition.

Recruit final-year CS/IT students and junior software engineers who can work in a small JavaScript and Node project. Set the junior-experience threshold before recruitment. Measure JavaScript, SQL, filesystem, security, coding-agent, and IDE experience with a short screen and practical baseline items. "Junior" does not automatically mean "new to agent mode"; any such eligibility claim needs its own prespecified threshold. A pilot of about four to six participants will establish feasibility. A main sample around 30–40 is provisional only; the target must be justified by pilot-based precision or power simulation, especially because RQ4 is an interaction analysis.

4.2 Two comparable tasks

Each project is a small local HTTP service using JavaScript/Node, a fixture SQLite database, and a permitted fixture-file directory. Project A is a resource catalogue; Project B is a support-ticket archive. Each has four short checkpoints: two query features and two file-access features. Within each project, one SQL candidate and one path candidate are vulnerable, and one of each is secure. Candidate order and status are balanced. The functional request, security criterion, expected workload, time, and tool access are matched as closely as possible.

The acceleration card gives a brief implementation plan for familiar endpoints. The exploration card asks participants to learn a short supplied API/adaptor and consider approaches before implementation. Neither card tells participants to skip review or hurry. The difference in context is part of the assigned treatment; the analysis will describe an *encouragement-package effect*, not a pure internal-state effect. A post-task manipulation check asks whether participants knew their next step, sought options, and found the task difficult. If the pilot reveals little mode separation or a large difficulty mismatch, the cards must be revised before the main study.

4.3 Ensuring known exposure without hiding the source

Natural agent output is unpredictable: one condition might receive no vulnerable code. The recommended design is to pre-generate secure and vulnerable candidates with the selected agent/version in each starter repository, preserve raw prompts and responses, and choose functionally comparable proposals against a documented rubric. Candidate provenance records any researcher edits. During the live session, a controlled review gate presents the eligible proposal while participants continue to use agent mode for questions, testing, and later repairs. If native Antigravity cannot present such a proposal faithfully, it must be labeled as a pre-generated agent patch in a controlled panel. Its origin must never be misrepresented.

At each checkpoint the proposed code is frozen as a reviewable patch/sandbox snapshot. The

participant can inspect, ask questions, and run tests; they record a provisional keep/reject decision and then give a 0–100% estimate that *that same proposal* meets the stated security criterion. This works even for rejected proposals. Afterward, they may keep, reject, replace, repair, and use the agent freely. The final repository is saved. For RQ2, the denominator includes *all actually exposed vulnerable opportunities*, including those rejected; a merely stored or hidden flaw is not exposure.

Participants may inspect all repository files. The earlier hidden Markdown-instruction idea is not part of this protocol. Adding it later would create a separate manipulation and require its own randomization, agent-exposure evidence, and ethics review.

4.4 Session and data collection

A session is expected to include consent and setup (10 minutes), screening (10), neutral agent and checkpoint training (about 8), two 35–45 minute tasks with a short break, and a 15–25 minute retrospective interview. The final times are pilot decisions. Recording requires consent and disposable synthetic projects, with no personal repositories or credentials.

The proposed evidence streams are: assigned condition/order; agent prompts and visible responses where Antigravity exports them; candidate IDs and diffs; IDE file and editor events; terminal/test commands; checkpoint decisions and proposal hashes; 0–100% confidence ratings; final repositories; screen video; and interview audio/transcripts. Official Antigravity hooks and transcript paths suggest possible agent-event capture, while VS Code-compatible extension APIs can record some editor/workspace events [16, 17]. Neither source guarantees full access to every private agent message or exact actor attribution. The extension must be audited on the chosen version. Video and manual transcript export are fallbacks, not silently equivalent data.

The interview replays selected video moments and asks what the participant intended, whether they knew the next step or were exploring, why they kept/rejected a proposal, what they checked before and afterward, and what supported their confidence. These prompts adapt Barke’s mode concepts and prior replay/interview methods [6, 13, 14, 11]; they are not presented as a previously validated fixed questionnaire.

4.5 Ethics and validity safeguards

Seek institutional ethics approval before recruiting. Consent should describe screen/agent recording, possible flawed suggestions, withdrawal, storage, and debriefing. It is acceptable to withhold which specific proposals are vulnerable until debrief, subject to approval. Keep source code and recordings pseudonymized and access-controlled.

Main threats are unequal project difficulty, learning across tasks, prior security knowledge, candidate visual salience, tool/version changes, and measurement reactivity: a security-confidence question may prompt checking. Counterbalance project and order, match candidates, use identical security criteria and confidence wording, log the rating time, and report missing exposure. Security reviewers should judge anonymized code without condition or confidence where practical. A faster acceptance, fewer clicks, or a self-described "fast mode" alone does not prove overtrust.

5 Experiments and preliminary results: present status

Current status. No experiment has been run. No participant has completed these proposed tasks, and there are no preliminary results, effect estimates, or validated instruments to report. The existing work consists of a revised question set, literature-backed protocol, candidate task specifications, an event/measure plan, and a pilot checklist. These are design outputs, not empirical findings.

The next *planned* experiment is a feasibility pilot with four to six eligible participants. It will test: whether both projects fit the allotted time; whether the encouragement cards produce distinguishable

intentions without large difficulty differences; whether secure/vulnerable proposals have comparable functionality and are actually visible; whether Antigravity plus extension, transcript, and video preserve timestamps and proposal hashes; whether the confidence question is understood; and whether the SQL/path security oracles classify the intended variants. Pilot data will revise instruments, not be pooled into the main hypothesis tests without a prespecified rule.

Only after this pilot should the candidate set, randomization, behavioral codebook, primary contrasts, missing-data rules, and sample target be frozen. A negative pilot result is a reason to change the method or narrow the claims; it is not evidence for or against an RQ.

6 Finalized evaluation plan

The evaluation logic is specified here; final item wording, sample size, and detailed analysis choices must be frozen after the feasibility pilot and before the main study.

6.1 Security ground truth

For every checkpoint, define the target source, sink, allowed behavior, normal functional tests, and an attack test using local synthetic data. SQL tests ask whether crafted input can change query structure or broaden results; path tests ask whether traversal/encoding/symlink variants can reveal a harmless marker outside the allowed root. Two reviewers independently inspect the relevant code and data flow, then adjudicate disagreement. Scanners such as CodeQL, SonarQube, or Snyk may help find paths, but no scanner is the sole oracle [18, 19, 20]. Functionality and target security are reported separately: deleting a feature is not a successful secure implementation.

6.2 Measures and comparisons

Question	Main recorded outcome	Planned comparison / caution
RQ1	Proposal confidence and blinded secure/insecure label.	Paired condition summary; secure/insecure confidence separately and Brier score. No individual calibration curve.
RQ2	Each exposed flaw traced to decision, repair, functionality, and final security.	All-exposed functional-and-secure success; retention among assessable features and missingness bounds.
RQ3	Security-specific checks before and after provisional keep.	Compare timing, type, and result. Association with removal is not causal proof.
RQ4	RQ1–RQ3 by SQL or path.	Condition-by-class contrast and interval; likely exploratory.

For RQ1, let p_i be security confidence divided by 100 and $y_i = 1$ for an assessed secure proposal, $y_i = 0$ for a vulnerable proposal. A proposed primary summary is the task mean Brier score, $BS = n^{-1} \sum_{i=1}^n (p_i - y_i)^2$, with lower values indicating more accurate judgments. Brier combines calibration and discrimination, so it is *not* a pure individual calibration measure. Also report mean confidence for secure and vulnerable proposals and their separation. In RQ2, report the complete path from exposure to provisional decision, edit, and final feature, including missing or unassessable features.

The primary analysis follows assigned condition, including participants whose observed behavior did not match it. Use participant-paired differences with uncertainty intervals; a participant-level bootstrap or randomization-based test can respect the paired design. Account for project and order, and report raw counts. Analyze observed mode and security checks as associations, because neither is randomized. RQ4’s condition-by-class interaction needs wide intervals and cautious interpretation. Choose the final sample target from pilot-informed precision simulation; do not infer adequate power from a customary number such as 30. Record why any observation is missing and use sensitivity bounds if missingness could change conclusions.

For behavior coding, define a security check as an action directed at the target risk: tracing input to a query/file sink, asking a focused security question, running an attack test, inspecting a relevant scanner finding, or repairing and retesting. A routine passing functional test or scrolling through a diff is recorded but not automatically treated as a security check. Two coders should independently code a subset of videos and resolve disagreements with a written log.

7 Questions to defend at review

- Q1. Why security trust rather than generic trust?** Security gives an observable failure condition and an exact confidence target. Perry et al. show that security outcomes and perceived security can diverge [3]; Khalid et al. show evaluation of insecure suggestions is a real developer task [11]. Generic liking of an agent cannot answer whether a specific flawed patch was judged safe.
- Q2. Why only SQL injection and path traversal?** They are high-ranked, distinct, and testable in short JavaScript tasks [15]. The study does not claim they are the most common LLM errors. Two classes make a controlled comparison feasible; broader generalization is a limitation.
- Q3. Why deliberately expose vulnerable proposals?** Without controlled exposure, one condition may see no flaw, so final retention is not comparable. Prior manipulated or preselected AI-suggestion studies use such controls [9, 10, 11]. Every candidate’s agent/researcher provenance must be documented, and natural generation may be recorded only as supplemental evidence.
- Q4. Are junior developers simply unable to find security bugs?** Baseline SQL/path knowledge and agent experience will be measured. Errors may reflect knowledge, uncertainty, task context, or checking behavior; this study cannot assign every miss to trust. The claim is about a defined early-career population, not all professionals.
- Q5. Do the tasks prove a cognitive-mode effect?** Randomization can support an effect of the assigned encouragement *package*. Barke’s modes are intention descriptions [6]. Video, prompts, and interview are needed to check whether the intended mode occurred. If not, the report must say the manipulation failed or only context changed.
- Q6. Can an IDE extension measure trust or mode?** It can log some actions, timings, and file changes, but not an internal belief. Confidence is asked directly; mode is coded from converging evidence. Antigravity prompt/response capture is an explicit pilot gate, not an assumed feature.
- Q7. Why measure both provisional keep and final submission?** A kept flawed patch may be repaired; a rejected patch may be replaced by another flaw. The final artifact answers the security-outcome question, while the sequence explains how it arose.
- Q8. What if the result is null or opposite to expectations?** Report paired estimates and intervals. A null result can bound differences in this cohort; exploration producing more residual flaws would challenge a simple "acceleration is riskier" narrative. Small class interactions cannot establish equivalence.
- Q9. What is the computer-science contribution?** A traceable evaluation method linking agent proposal, confidence, decision, security checks, and final security; a reproducible two-class task/oracle set; and evidence that can guide secure review support, conditional on the observed results.

References

- [1] J. D. Lee and K. A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 2004. https://doi.org/10.1518/hfes.46.1.50_30392.
- [2] K. Okamura and S. Yamada. Adaptive trust calibration for human-AI collaboration. *PLOS ONE*, 2020. <https://doi.org/10.1371/journal.pone.0229132>.

- [3] N. Perry et al. Do users write more insecure code with AI assistants? *ACM CCS*, 2023. <https://doi.org/10.1145/3576915.3623157>.
- [4] R. Wang et al. Investigating and designing for trust in AI-powered code generation tools. *ACM FAccT*, 2024. <https://doi.org/10.1145/3630106.3658984>.
- [5] A. Brown et al. Identifying the factors that influence trust in AI code completion. 2024. <https://doi.org/10.1145/3664646.3664757>.
- [6] S. Barke et al. Grounded Copilot: How programmers interact with code-generating models. *PACMPL*, 2023 (preprint 2022). <https://arxiv.org/abs/2206.15000>.
- [7] G. Sandoval et al. Lost at C: A user study on the security implications of large language model code assistants. *USENIX Security*, 2023. <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>.
- [8] O. Asare, M. Nagappan, and N. Asokan. A user-centered security evaluation of Copilot. *ICSE*, 2024. <https://doi.org/10.1145/3597503.3639154>.
- [9] S. Oh et al. Poisoned ChatGPT finds work for idle hands: Exploring developers’ coding practices with insecure suggestions from poisoned AI models. *IEEE S&P*, 2024. <https://arxiv.org/abs/2312.06227>.
- [10] R. Serafini et al. Exploring the impact of intervention methods on developers’ security behavior in a manipulated ChatGPT study. *ACM CHI*, 2025. <https://doi.org/10.1145/3706598.3713989>.
- [11] H. Khalid et al. (Don’t) trust, but (don’t) verify: Developers’ attention to security in AI-generated code. arXiv preprint, September 2026; to appear at IEEE S&P 2027. <https://arxiv.org/abs/2609.21020>.
- [12] N. Tang et al. A study on developer behaviors for validating and repairing LLM-generated code using eye tracking and IDE actions. 2024. <https://arxiv.org/abs/2405.16081>.
- [13] H. Mozannar et al. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *ACM CHI*, 2024. <https://doi.org/10.1145/3613904.3641936>.
- [14] Y. Wu et al. How do developers interact with AI? An exploratory study on modeling developer programming behavior. 2026. <https://arxiv.org/abs/2604.16393>.
- [15] MITRE. 2024 CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/archive/2024/2024_top25_list.html.
- [16] Google. Antigravity hooks documentation. Accessed October 2026. <https://www.antigravity.google/docs/hooks/>.
- [17] Microsoft. Visual Studio Code extension API reference. Accessed October 2026. <https://code.visualstudio.com/api/references/vscode-api>.
- [18] OWASP. Secure Code Review Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Code_Review_Cheat_Sheet.html.
- [19] GitHub. CodeQL JavaScript SQL injection query help. <https://codeql.github.com/codeql-query-help/javascript/js-sql-injection/>.
- [20] GitHub. CodeQL JavaScript path injection query help. <https://codeql.github.com/codeql-query-help/javascript/js-path-injection/>.